MPOO 2027-1 · GRUPO 06

Parcial 1 · correcciones

Versión A del examen del 7 de octubre. Para cada pregunta vienen el enunciado, tu respuesta tal como la enviaste en el portal y su corrección. La calificación no va aquí: la ves con el profesor.

Antes de empezar. Este archivo lo subió el profesor a tu rama. Trae el cambio a tu computadora antes de tu siguiente push, o tu push va a fallar:

```
git checkout entregas_velasco_iker
git pull --no-rebase
```

Qué tienes que hacer. En hojas, **a mano**, copia cada pregunta y escribe debajo tu respuesta corregida, usando la corrección que viene aquí. Donde la corrección ya te da la salida o el código, no basta con copiarlo: escribe **por qué** sale así, con la tabla de seguimiento de las variables, la causa del error o lo que devuelve tu método en cada renglón de la tabla. Debajo de esas preguntas viene qué escribir. Escanea las hojas o tómales foto, júntalas en un PDF y súbelo a esta misma carpeta, `entregas/velasco_iker/examen/p1/`, con tu push de siempre.

Pregunta 0.1 · Preguntas abiertas

Pregunta

Contesta con tus palabras, en una a tres oraciones. Puedes apoyarte en un ejemplo pequeño de código

¿Qué diferencia hay entre una clase y un objeto? Usa la idea de plano e instancia.

Tu respuesta

```
La clase es el molde que define las caracteristicas de un objeto , el objeto es una instancia creada a partir de una clase.
```

Corrección. Bien: la clase es el molde y el objeto es una instancia creada a partir de ella. Falta decir que cada objeto tiene su propio estado, es decir, sus propios valores de los atributos de instancia.

Pregunta 0.2 · Preguntas abiertas

Pregunta

¿Qué es la firma de un método y qué partes la forman? Explica si incluye el tipo de retorno.

Tu respuesta

```
La firma es el nombre y los parametros que tipo de valor es int boolean etc
```

Corrección. Bien: la firma la forman el nombre del método y sus parámetros, y mencionas el tipo de esos parámetros (int, boolean). Faltó precisar que cuentan los tipos de los parámetros en orden y contestar lo que se pedía explícitamente: el tipo de retorno no forma parte de la firma.

Pregunta 0.3 · Preguntas abiertas

Pregunta

Compara la anatomía de una variable local y un atributo de instancia: dónde se declaran, valor inicial y cuánto viven.

Tu respuesta

```
una variable local existe solo en donde es creada y los atributos de instancia solo existen mientras son utilizados
```

Corrección. Es correcto que la local existe solo donde se declara, aunque falta decir que debe recibir un valor antes de leerse. Lo del atributo es incorrecto: se declara en la clase fuera de los métodos, recibe un valor por defecto y dura lo que vive el objeto, no 'mientras se utiliza'.

Pregunta 0.4 · Preguntas abiertas

Pregunta

¿Qué significa null en una referencia? Explica qué ocurre al llamar un método sobre esa referencia.

Tu respuesta

```
Null es parecido al 0 al llamar un metodo asi no devuelve nada
```

Corrección. Incorrecto: null no es parecido a 0, y llamar un método sobre esa referencia no 'deja de devolver algo'. null significa que la referencia no apunta a ningún objeto; llamar un método de instancia sobre ella hace que el programa truene al ejecutarse (NullPointerException), aunque asignar null no falla.

Pregunta 0.5 · Preguntas abiertas

Pregunta

¿Cuándo conviene usar for-each para recorrer un arreglo y cuándo necesitas un recorrido con índice?

Tu respuesta

```
el ciclo conviene cuando tienes varios elementos y necesitas ir uno a uno ,cuando tienes indice
```

Corrección. Se asoma la distinción entre ir 'uno a uno' y usar índice, pero no se explica cuándo conviene cada uno. for-each sirve cuando solo se necesitan los valores (por ejemplo, para sumarlos); el for con índice hace falta cuando se necesita la posición o cuando hay que reemplazar elementos.

R1 · ¿Qué imprime?

Pregunta

Escribe la salida exacta, una línea por renglón. El fragmento está en un main válido.

```
StringBuilder ruta = new StringBuilder("S");
StringBuilder alias = ruta;
for (int i = 1; i <= 3; i++) {
    alias.append(i / 2);
```

```
    System.out.println(ruta);
}
String texto = "S" + 2 + 1;
System.out.println(texto);
double mitad = (double) (7 / 2);
System.out.println(mitad);
```

Tu respuesta

```
3
V
3.5
```

Corrección. Solo la última línea tiene una parte reconocible: 3.5 muestra que se imprime un double, pero 7/2 da 3 antes del cast, así que imprime 3.0. Escribió tres líneas en vez de cinco y las otras no coinciden. La salida correcta es S0 / S01 / S011 / S21 / 3.0: alias y ruta apuntan al mismo StringBuilder, i/2 agrega 0, 1 y 1, y la concatenación va de izquierda a derecha.

Qué escribir a mano. La salida ya la tienes en la corrección. Escribe por qué sale así: una tabla de seguimiento con el valor de cada variable en cada vuelta y en cada println.

R2 a) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
double medida = 6.8;
int unidades = medida;
System.out.println(unidades);
```

Tu respuesta

Clasificación

```
Funciona
```

Causa o salida

```
Primero usa double que sirve para numeros con decimales como el 6.8 y despues lo cambia a
entero en todo caso si imprime solo imprime el 6 no recuerdo si causa un error
```

Corrección. La clasificación es incorrecta: no compila. No es que trunque a 6: Java no convierte double a int de forma implícita, así que hace falta un cast explícito como (int) medida.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 b) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
int[] canales = {2, 5, 8};
int indice = canales.length;
System.out.println(canales[indice]);
```

Tu respuesta

Clasificación

Funciona

Causa o salida

2,5,8

> **Corrección.** Incorrecto: compila pero truena. length vale 3 y el último índice válido es 2, así que canales[3] lanza ArrayIndexOutOfBoundsException sin imprimir nada antes.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R2 c) · ¿Compila, truena o funciona?

Pregunta

Cada inciso va en un main independiente. Indica no compila, compila pero truena o funciona y da la causa en una frase. Si funciona, escribe la salida (separa líneas con /).

```
String codigo = new String("F2");
System.out.println(codigo == "F2");
System.out.println(codigo.equals("F2"));
```

Tu respuesta

Clasificación

No compila

Causa o salida

== y equals son diferentes uno lo usamos para asignacion y el otro para comparacion

> **Corrección.** Incorrecto: sí compila y funciona, e imprime false / true. == no asigna (eso lo hace =): compara si dos referencias son el mismo objeto, y new String crea un objeto distinto; equals compara el contenido.

Qué escribir a mano. La clasificación ya la tienes en la corrección. Escribe por qué: qué línea causa el error y qué regla de Java lo provoca o, si funciona, cómo se llega a esa salida.

R3 · Modelado: base, reto y justificación

Pregunta

Cada multímetro conserva su inventario y nace sin responsable. Se cuenta el total de prestados. Escribe prestar(persona) y una consulta del responsable. Supón que solo se llama prestar con un aparato disponible y un nombre válido.

Base. Declara al menos dos atributos de instancia con tipo, uno static y una constante final (¿también static?). Completa constructor y dos métodos: cambio y consulta.

Reto. ¿La cuenta de prestados pertenece a cada aparato o a todos? ¿Qué dato nunca cambia?

Justifica. Una línea por decisión: tipo, instancia/clase, final o mutable, constructor y métodos.

Cuerpos de 1 a 3 líneas, sin estructuras de control ni main. Hay más de un modelo correcto. Un elemento sin justificar vale la mitad.

Tu respuesta

Clase

```
La cantidad de multimetros siempre sera constante al final
debemos declarar el objeto multimetro y nombre  hacemos un ciclo que verifique la fantidad de
mu8ltimetros disponibles segun lo que ya prestamos y los que tenemos en total si multimetro
menor o igual a la cantidad total entonces pregunta el nombre que vamos a guardar en esa
variable para saber quien lo tiene,una vez prestado se resta a la cantidad de multimetros
disponibles
```

Justificaciones

Sin respuesta.

> **Corrección.** Contestó en prosa, sin clase, atributos con tipo, constructor ni justificaciones, y propone un ciclo, que el enunciado prohíbe. Se reconocen la idea de guardar quién tiene el aparato y la de llevar una cuenta que cambia al prestar, y como dato fijo propone la cantidad total. El modelo esperado lleva: inventario String final de instancia, responsable String mutable de instancia, prestados static int compartido, un constructor que inicializa con this, y los métodos prestar (asigna el responsable y hace prestados++) y consultarResponsable.

R4 · Código con repetición

Pregunta

Un equipo guarda sus folios como enteros no negativos. Devuelve cuántos dígitos tiene folio usando while y división entre 10. El folio 0 tiene un dígito. No conviertas el número a texto.

```
static int contarDigitos(int folio)
```

Ejemplos (argumentos → retorno):

4072 → 4 · 0 → 1 · 8 → 1

Escribe el método completo. No imprimas ni leas del teclado.

Tu respuesta

Sin respuesta.

> **Corrección.** Sin respuesta.

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.

R5 · Código con selección

Pregunta

Clasifica la carga de un equipo. Usa switch clásico sobre carga / 10. Puedes validar antes con if. Devuelve el texto de la tabla. Los límites están incluidos. No imprimas ni leas del teclado.

```
static String nivelCarga(int carga)
```

Regla o valor recibido	Texto devuelto
Menor que 0 o mayor que 50	"INVALIDA"
0 a 19	"BAJA"
20 a 39	"MEDIA"
40 a 50	"ALTA"

Tu respuesta

```
en este debemos hacer un if else con case en el que va ir revisando la cantidad de la variable carga usando los parametros if carga >= 0 o <= 50 entonces print ln invalido y asi con cada uno de los parametros al final cualquier cosa que no sea NUMERO devolvemos Null
```

Corrección. No escribió código; describe en prosa una selección con if y case. La validación está invertida (0 a 50 lo marca como inválido), imprime en vez de devolver y devuelve null; no hay cabecera, no hay switch sobre carga/10 ni agrupación de cocientes. Lo esperado: if (carga < 0 || carga > 50) return "INVALIDA"; luego switch (carga / 10) con case 0 y 1 devolviendo "BAJA", case 2 y 3 "MEDIA", y default "ALTA".

Qué escribir a mano. El método corregido completo y, para cada ejemplo o renglón de la tabla del enunciado, qué devuelve tu método y por qué.